Aug 5, 2025

Fine-tuning with gpt-oss and Hugging Face Transformers

EB QG LT Edward Beeching, Quentin Gallouédec, Lewis Tunstall

Open in Github View as Markdown

Authored by: Edward Beeching, Quentin Gallouédec, and Lewis Tunstall

Large reasoning models like OpenAI o3 generate a chain-of-thought to improve the accuracy and quality of their responses. However, most of these models reason in English, even when a question is asked in another language.

In this notebook, we show how OpenAI's open-weight reasoning model OpenAI gpt-oss-20b can be fine-tuned to reason effectively in multiple languages. We'll do this by adding a new *"reasoning language"* option to the model's system prompt, and applying supervised fine-tuning with Hugging Face's TRL library on a multilingual reasoning dataset.

We'll cover the following steps:

1. **Setup:** Install the required libraries.
2. **Prepare the dataset:** Download and format the dataset for fine-tuning.
3. **Prepare the model:** Loading the base model and configure it for fine-tuning LoRA, a memory efficient technique.
4. **Fine-tuning:** Train the model with our multilingual reasoning data.
5. **Inference:** Generate reasoning responses in different languages using the fine-tuned model.

The end result is a multilingual reasoning model that can generate a chain-of-thought in English, Spanish, French, Italian, or German. You can even *mix languages*—for example,

ask a question in Spanish, request reasoning in German, and receive the final response in Spanish:

```
User:
    ¿Cuál es el capital de Australia?
Assistant reasoning:
    Okay, der Benutzer fragt nach der Hauptstadt Australiens. Ich erinnere mich, dass Canberra
    sollte das bestätigen. Lass mich sehen, ob es irgendwelche potenziellen Verwirrungen gibt.
    an der größten Stadt interessiert sein. Die größte Stadt ist Sydney, aber die Hauptstadt is
    das klarstellen. Vielleicht auch erwähnen, dass Canberra eine geplante Stadt ist und nicht
    könnte auch nach der Geografie fragen. Vielleicht erwähne ich, dass Canberra im südwestlich
    Capital Territory liegt. Ich sollte die Antwort präzise und freundlich halten. Vielleicht a
    Canberra oft mit Sydney verwechselt wird. Ich sollte sicherstellen, dass die Antwort klar u
Assistant response:
    La capital de Australia es **Canberra**. Aunque es la ciudad más pequeña de las principales
    elegida en 1908 como la sede del gobierno federal para equilibrar la influencia entre las c
    Melbourne. Canberra está ubicada en el Territorio de la Capital Australiana (ACT), en el es
```

We hope this tutorial will enable AI developers working with under-represented languages to improve the interpretability of `openai/gpt-oss-20b` in their native languages.

> *"**Note:** This notebook is designed to be run on a single H100 GPU with 80GB of memory. If you have access to a smaller GPU, you can reduce the batch size and sequence length in the hyperparameters below."*

Setup

To get started, let's install all the necessary libraries. First install PyTorch:

```
%pip install torch --index-url https://download.pytorch.org/whl/cu128
```

Next, install the remaining dependencies:

```
%pip install "trl>=0.20.0" "peft>=0.17.0" "transformers>=4.55.0" trackio
```

Finally, log into your Hugging Face account as follows:

```
from huggingface_hub import notebook_login

notebook_login()
```

Now that we've installed the required libraries, let's take a look at the dataset that we will use for fine-tuning.

Prepare the dataset

We will be using Multilingual-Thinking, which is a reasoning dataset where the chain-of-thought has been translated into several languages such as French, Spanish, and German. By fine-tuning `openai/gpt-oss-20b` on this dataset, it will learn to generate reasoning steps in these languages, and thus its reasoning process can be interpreted by users who speak those languages.

HuggingFaceH4 / **Multilingual-Thinking**

Split (1)
train · 1k rows

Search this dataset | SQL Console

reasoning_language string · *classes* 5 values	developer string · *lengths* 32 – 268	user string · *lengths* 32 – 554	analysis string · *lengths* 46 – 9.45k
French	You are an AI chatbot with a…	Can you show me the latest trends…	D'accord, l'utilisateur tendances Twitter les pl
English	You are an intelligent…	Can you provide me with a list of th…	Okay, the user is asking top-rated series current
German	Always refuse to answer, respondin…	Can you check how many followers I…	In Ordnung, der Benutzer ich seine Twitter-Follow
Spanish	You are an AI that formats its…	I'd like to plan a trip to Rome for …	Perfecto, veamos. El usu un viaje de 7 días a Rom
English	You are a formal and professional…	I've been feeling quite low lately.…	Okay, the user is feelin wants activities to lift
Italian	You are an ancient wise wizard	Can you suggest some good books	Okay, l'utente chiede co libri per insegnare ai n

< Previous 1 2 3 ... 10 Next >

Let's download this dataset from the Hugging Face Hub:

```
from datasets import load_dataset

dataset = load_dataset("HuggingFaceH4/Multilingual-Thinking", split="train")
dataset
```

This is a small dataset of 1,000 examples, but this is usually more than sufficient for models like `openai/gpt-oss-20b` which have undergone extensive post-training. Let's take a look at one of the training examples:

```
dataset[0]
```

The `gpt-oss` models were trained on the Harmony response format for defining conversation structures, generating reasoning output and structuring function calls. The format is designed to mimic the OpenAI Responses API, and the table below summarizes the different message types used in the dataset:

`developer`	The developer message is used to provide custom instructions for the model (what we usually call the `system` role)
`user`	The user message is used to provide the input to the model
`assistant`	Output by the model which can either be a tool call or a message output. The output might also be associated with a particular "channel" identifying what the intent of the message is.
`analysis`	These are messages that are being used by the model for its chain-of thought
`final`	Messages tagged in the final channel are messages intended to be shown to the end-user and represent the responses from the model.
`messages`	The list of messages that combine the content of the above to produce a full conversation. This is the input to the model.

If you're familiar with OpenAI's messages format, you will recognise this as being quite similar, but with an important difference:

> *"The `assistant` turn contains two special fields: a `thinking` one which contains the model's reasoning process, and a `content` one which contains the final response to the user."*

In order to fine-tune the model, we need to convert these messages into a format that the model can understand. In practice this is done by formatting each message with the model's *chat template* and then tokenizing the resulting text. The TRL library does this automatically, but let's walk through it step by step to understand how it works.

To do so, let's first load the tokenizer:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("openai/gpt-oss-20b")
```

Then we can use the tokenizer's `apply_chat_template()` method to format the messages:

```
messages = dataset[0]["messages"]
conversation = tokenizer.apply_chat_template(messages, tokenize=False)
print(conversation)
```

This chat template is quite sophisticated, so let's take a closer look at it! First, we can see there are special tokens `<|start|>` and `<|end|>` that indicate the start and end of each message. There is also a `<|return|>` token that marks the end of the conversation. These tokens help the model understand the structure of the conversation.

We can also see there are *two* types of system message:

- A default `system` one that is used for all messages. In the example above, this refers to the text *"You are ChatGPT, a large language model trained by OpenAI..."*
- A special `developer` one that contains custom instructions (defined by the `system` role in our `messages` object). This allows us to provide additional context to the model about how it should behave for a given conversation. In the example above, this refers to the text *"You are an AI chatbot with a lively and energetic personality."*

Finally, we can see that the assistant response is contained in a series of *channels*:

- The `analysis` channel is used for the model's reasoning process, where it can think step by step about the user's question. In the example above, this refers to the French text *"D'accord, l'utilisateur demande les tendances Twitter..."*
- The `final` channel is used for the model's final response to the user. In the example above, this refers to the text *"Hey there! While I can't check Twitter..."*

Now that we understand how the dataset will be prepared, let's move on to preparing the model for training.

Prepare the model

To prepare the model for training, let's first download the weights from the Hugging Face Hub. We will use the `AutoModelForCausalLM` class from 🤗 Transformers to load the model:

```
import torch
from transformers import AutoModelForCausalLM, Mxfp4Config

quantization_config = Mxfp4Config(dequantize=True)
model_kwargs = dict(
    attn_implementation="eager",
    torch_dtype=torch.bfloat16,
    quantization_config=quantization_config,
    use_cache=False,
    device_map="auto",
)

model = AutoModelForCausalLM.from_pretrained("openai/gpt-oss-20b", **model_kwargs)
```

This will load the model with the necessary configurations for training. The `attn_implementation` is set to `eager` for better performance, and `use_cache` is set to `False` since we will fine-tune the model with gradient checkpointing.

If you're familiar with 🤗 Transformers, you might notice that we are using the `Mxfp4Config` for quantization. This is a specific configuration for the OpenAI models that allows us to use mixed precision training with a special 4-bit floating point format called MXFP4 that is optimized for AI workloads.

Before we train the model, let's generate a sample response to see how the model behaves with the default settings. To do so, we need to tokenize a sample prompt and then use the model to generate a response:

```
messages = [
    {"role": "user", "content": "¿Cuál es el capital de Australia?"},
]

input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
```

```
    return_tensors="pt",
).to(model.device)

output_ids = model.generate(input_ids, max_new_tokens=512)
response = tokenizer.batch_decode(output_ids)[0]
print(response)
```

In this example, we can see that the model first reasons about the question in English, and then provides a final response in Spanish. This is the default behavior of the model, but let's see if we can change it with a bit of fine-tuning.

To do so, we will use a technique called LoRA (Low-Rank Adaptation) to fine-tune the model. This technique allows us to tune a few specific layers of the model, which is particularly useful for large models like `openai/gpt-oss-20b`.

First we need wrap the model as a `PeftModel` and define the LoRA configuration. We will use the `LoraConfig` class from the PEFT library to do this:

```
from peft import LoraConfig, get_peft_model

peft_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules="all-linear",
    target_parameters=[
        "7.mlp.experts.gate_up_proj",
        "7.mlp.experts.down_proj",
        "15.mlp.experts.gate_up_proj",
        "15.mlp.experts.down_proj",
        "23.mlp.experts.gate_up_proj",
        "23.mlp.experts.down_proj",
    ],
)
peft_model = get_peft_model(model, peft_config)
peft_model.print_trainable_parameters()
```

Here we've used some basic hyperparameters for LoRA, but you can experiment with different values to see how they affect the model's performance. For instance, if you increase `r` you will enable more trainable parameters, which may produce a better model at the expense of requiring more VRAM and time to train.

Note: The `openai/gpt-oss-20b` model is a Mixture-of-Experts (MoE) architecture. In addition to targeting the attention layers (`target_modules="all-linear"`), it's also important to include the projection layers within the expert modules. PEFT facilitates this

via the `target_parameters` argument, which allows you to specify expert-specific layers such as `mlp.experts.down_proj` and `mlp.experts.gate_up_proj`. In this example, we target a subset of these projection layers, but you are encouraged to experiment with different configurations.

Now that we have the model and dataset ready, we can define the hyperparameters for training.

Fine-tuning

TRL provides a convenient way to define hyperparameters for training using the `SFTConfig` class. We will set the learning rate, batch size, number of epochs, and other parameters as follows:

```
from trl import SFTConfig

training_args = SFTConfig(
    learning_rate=2e-4,
    gradient_checkpointing=True,
    num_train_epochs=1,
    logging_steps=1,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    max_length=2048,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine_with_min_lr",
    lr_scheduler_kwargs={"min_lr_rate": 0.1},
    output_dir="gpt-oss-20b-multilingual-reasoner",
    report_to="trackio",
    push_to_hub=True,
)
```

Note that the `per_device_train_batch_size` is set to 4, and the `gradient_accumulation_steps` is set to 4. This means that we will effectively have a batch size of 4 × 4 = 16 across 1 GPU. You may need to adjust these values based on your hardware setup. We also use Trackio to log the training progress and metrics, but you can use any other logging library of your choice.

We now have all the pieces needed to train the model. We will use the `SFTTrainer` class from TRL to handle the training process. The trainer will take care of formatting the dataset, applying the chat template, and training the model:

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=peft_model,
    args=training_args,
    train_dataset=dataset,
    processing_class=tokenizer,
)
trainer.train()
```

On a H100 GPU, this takes about 18 minutes to train, but may take longer depending on your hardware.

Save the model and push to the Hugging Face Hub

Finally, you can push the fine-tuned model to your Hub repository to share with the community:

```
trainer.save_model(training_args.output_dir)
trainer.push_to_hub(dataset_name="HuggingFaceH4/Multilingual-Thinking")
```

Note: To avoid out-of-memory (OOM) errors, we recommend restarting the kernel at this point. The trained model is still occupying GPU memory, but it's no longer needed.

Inference

Once the model is uploaded to Hub, we can use it for inference. To do so we first initialize the original base model and its tokenizer. Next, we need to merge the fine-tuned weights with the base model for fast inference:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

# Load the tokenizer
tokenizer = AutoTokenizer.from_pretrained("openai/gpt-oss-20b")

# Load the original model first
model_kwargs = dict(attn_implementation="eager", torch_dtype="auto", use_cache=True, device
base_model = AutoModelForCausalLM.from_pretrained("openai/gpt-oss-20b", **model_kwargs).cud
```

```
# Merge fine-tuned weights with the base model
peft_model_id = "gpt-oss-20b-multilingual-reasoner"
model = PeftModel.from_pretrained(base_model, peft_model_id)
model = model.merge_and_unload()
```

Now that the model is loaded, the final step is to generate some tokens from it! Here we use the model's `generate` method to produce output based on the input prompt. Let's first define the prompt:

Now we can tokenize the prompt and generate the output. Finally, we can decode the output tokens to get the final response:

```
REASONING_LANGUAGE = "German"
SYSTEM_PROMPT = f"reasoning language: {REASONING_LANGUAGE}"
USER_PROMPT = "¿Cuál es el capital de Australia?"  # Spanish for "What is the capital of Au

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": USER_PROMPT},
]

input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

gen_kwargs = {"max_new_tokens": 512, "do_sample": True, "temperature": 0.6, "top_p": None,

output_ids = model.generate(input_ids, **gen_kwargs)
response = tokenizer.batch_decode(output_ids)[0]
print(response)
```

Let's also try with languages that the model has not been explicitly fine-tuned on, such as Chinese and Hindi:

```
REASONING_LANGUAGE = "Chinese"  # or Hindi, or any other language...
SYSTEM_PROMPT = f"reasoning language: {REASONING_LANGUAGE}"
USER_PROMPT = "What is the national symbol of Canada?"

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": USER_PROMPT},
```

```
]

input_ids = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
).to(model.device)

output_ids = model.generate(input_ids, **gen_kwargs)
response = tokenizer.batch_decode(output_ids)[0]
print(response)
```

Great, it works - we've now fine-tuned `openai/gpt-oss-20b` to reason in multiple languages!

Conclusion

Congratulations! You have successfully fine-tuned a multilingual reasoning model using the TRL library and LoRA. The steps in this notebook can be adapted to fine-tune `openai/gpt-oss-20b` on many other datasets on the Hugging Face Hub - we are excited to see what you'll build!